ChatGPT

Bluesky AT Protocol API – Comprehensive Reference Guide

This guide provides a detailed reference for Bluesky's AT Protocol APIs, covering all current functionality. It is organized into sections for authentication, record management (posts, profiles, follows, likes), feed reading and custom feed generation, moderation tools, content labels/filtering, lexicon schemas, advanced features, and best practices. All endpoints are identified by their Lexicon NSIDs (names like `com.atproto.*` or `app.bsky.*`) and are accessed via HTTP XRPC calls (generally at paths like `/xrpc/<NSID>`). Examples are provided in TypeScript using Bluesky's official `@atproto/api` SDK.

Note on API Hosts: Many *public* read-only endpoints can be queried against Bluesky's AppView service (`https://public.api.bsky.app`) without authentication. Authenticated or user-specific requests are typically sent to the user's own PDS (Personal Data Server) host (e.g. `https://bsky.social` for Bluesky) [1] [2]. The client libraries handle service discovery (resolving a user's handle to their PDS) automatically.

Authentication

Bluesky supports two authentication mechanisms: a legacy *session token* system (using App Passwords and JWTs) and a newer OAuth 2.1-based system. As of now, most API usage is via the session token system, but OAuth is the recommended path moving forward [3].

- **Session Endpoint Authentication (Legacy):** Clients can log in by calling `com.atproto.server.createSession` with the user's handle (or email) and password (often an *App Password* rather than the main password) [3]. This returns a JSON object containing an `accessJwt` (short-lived access token) and a `refreshJwt` (long-lived token) [4]. For example, using the TypeScript SDK:

```
import { BskyAgent } from '@atproto/api';
const agent = new BskyAgent({ service: 'https://bsky.social' });
await agent.login({ identifier: 'alice.bsky.social', password: 'app-
password-123' });
// The agent now holds accessJwt & refreshJwt for authenticated requests.
```

After login, the client includes the `Authorization: Bearer <accessJwt>` header on API calls. The access token expires after a few minutes, so the client should periodically refresh it by calling `com.atproto.server.refreshSession` with the `refreshJwt` [5]. The refresh endpoint returns a new access JWT (and potentially a new refresh token). Most SDKs handle token refresh automatically. The session can be terminated with `com.atproto.server.deleteSession` (revoking the tokens).

App Passwords: It's recommended to use an App Password (generated in the user's account settings) instead of the main account password for API use. App Passwords can be created/revoked by the user

and provide limited scope (e.g. no account deletion) [3]. The `createSession` login accepts the handle and app password just like a normal password.

- **OAuth 2.1 Authentication:** The AT Protocol implements an OAuth 2.1 flow for third-party apps to obtain user authorization without handling raw passwords [6] [3]. In the OAuth model, your app registers (dynamically) via a *Client Identifier* and uses standard authorization code flows. Users will authorize your app via the server's OAuth page, and your app receives an access token. This is the preferred approach for new integrations, offering fine-grained scopes and improved security (the user never gives the app their password) [7] [8]. Bluesky provides OAuth client libraries for TypeScript (`@atproto/oauth-client-*`) to streamline this process [9].

Example: A web app can initiate OAuth with the user's handle, which gets resolved to their PDS (using DNS or `.well-known` as per ATProto handle resolution). The user is redirected to their server's OAuth consent screen, and after approval the app receives an authorization code to exchange for tokens [8] [10]. The OAuth token can then be used as a Bearer token on API calls (similar to the JWT). The OAuth flow is beyond this guide's full scope, but developers should refer to the [ATProto OAuth spec and guides] [11] [12]. OAuth supports scopes (e.g. read:posts, write:posts, etc.), and tokens can be revoked or limited without affecting the user's primary credentials.

Endpoints Summary – Authentication:

Endpoint	Description
`com.atproto.server.createSession` **POST**	Login with handle & password (or App Password). Returns `{ accessJwt, refreshJwt }` for the session [4].
`com.atproto.server.refreshSession` **POST**	Refresh the access JWT using a valid refresh JWT [13]. Keeps the session alive without relogin.
`com.atproto.server.deleteSession` **POST**	Invalidate the current session (logout).
`com.atproto.server.createAppPassword` **POST**	Create a new App Password for the account (requires auth). *(Typically App Passwords are created via UI, but an API exists.)*
OAuth Endpoints (various)	OAuth 2.1 authorize and token endpoints (flow handled via PDS's OAuth service) [8]. Use instead of above for third-party apps.

Usage Tip: When using the official TypeScript SDK, the `BskyAgent` (or `AtpAgent`) will manage session tokens for you. After calling `agent.login(...)`, all subsequent `agent` API calls automatically include the `Authorization` header [14]. If tokens expire, the agent can refresh them. When using raw HTTP, remember to handle 401 responses by refreshing the token. Also note that the Bluesky AppView (`public.api.bsky.app`) does not accept authenticated writes; you must use the user's PDS hostname (e.g. `bsky.social`) for authenticated requests [1].

Record Management (Posts, Profiles, Follows, Likes)

Overview: In AT Protocol, user data is stored in a personal **repository** of **records**. Records are JSON objects defined by Lexicon schemas and grouped in **collections** (each collection has an NSID, such as

`app.bsky.feed.post` for posts). The protocol provides generic repository operations to create, update, and delete records, as well as read them. Common social features (creating posts, liking, following, etc.) are all implemented as records in the user's repo.

Creating and Managing Posts (Feed Records)

A *post* in Bluesky is a record of type `app.bsky.feed.post`. Posting is done via the generic repository write endpoint `com.atproto.repo.createRecord`. To create a post, you supply your repo DID or handle, the collection NSID, and the record contents. For example, using the TS SDK:

```
// Create a new post
await agent.post({
  text: 'Hello world! I posted this via the API.',
  createdAt: new Date().toISOString()
});
```

Under the hood, this calls `com.atproto.repo.createRecord` with `collection: "app.bsky.feed.post"` and a JSON record containing the post text and timestamp [15]. The response returns the URI of the new post (`at://<your DID>/app.bsky.feed.post/<rkey>`) and its CID (content hash) [16]. For example:

```
{
  "uri": "at://did:plc:abcd1234/app.bsky.feed.post/3kzxcvy…",
  "cid": "bafyreih…"
}
```

You can use the URI to reference the post elsewhere (e.g. in replies or reposts). The `rkey` (record key) in the URI is usually an auto-generated TID (timestamp-based ID) since the `app.bsky.feed.post` schema specifies `"key": "tid"`.

Posts may include optional fields defined in the lexicon, such as: - `reply` (to make the post a reply to another post, containing `root` and `parent` URIs/CIDs), - `embed` (to embed media or other records, e.g. images or quoted posts), and - facets for rich text (links or mention facets).

These follow the schemas in `app.bsky.feed.post` and related lexicons. For instance, a reply post will include a `reply` field with strong references to the parent and root posts in the thread [17] [18]. Likewise, images are uploaded via `com.atproto.repo.uploadBlob` and then included as an embed in the post record (with alt text) [19] [20].

To **delete** a post, use `com.atproto.repo.deleteRecord`, providing the `collection` and the `rkey` of the record to delete. Deletion permanently removes that record from your repo (though others who have seen it may have cached it or its CID). There is currently no in-place editing of a post's content – to "edit" you would delete and repost, or (in the future) utilize an edit history feature if introduced.

Profile Records (User Profile)

A user's profile (display name, bio description, avatar, etc.) is stored as a record `app.bsky.actor.profile` in their repository. By convention, this record has the unique key `"self"` (the lexicon uses a fixed key since one profile per account) [21]. To update your profile, you replace this record.

The TS SDK provides a convenient method `agent.updateProfile({...})` (or similar) to call the appropriate endpoint. Under the hood, updating a profile can be done with `com.atproto.repo.putRecord` (since the record already exists) – you supply the collection `app.bsky.actor.profile`, the `rkey: "self"`, and the new record value. The profile record schema includes fields like `displayName` (string), `description` (string), and optional `avatar` and `banner` (which are references to uploaded image blobs). For example:

```
await agent.api.com.atproto.repo.putRecord({
  repo: myDid,
  collection: 'app.bsky.actor.profile',
  rkey: 'self',
  record: {
    displayName: 'Alice Wonderland',
    description: 'Exploring AT Protocol',
    avatar: { cid: 'bafy...', mimeType: 'image/jpeg' }  // blob ref from
uploadBlob
  }
});
```

The `getProfile` and `getProfiles` endpoints (`app.bsky.actor.getProfile` / `getProfiles`) can retrieve profile records (including badges, follower count, etc.). Note that profiles (and all records in a user's repo) are public by default – any server or app can fetch another user's profile record via their DID or handle [2].

Follows and Unfollows

"Following" someone is represented by an `app.bsky.graph.follow` record in your repo. When you follow user X, your PDS creates a follow record with `{ subject: X's DID, createdAt: timestamp }`. This is done via `com.atproto.repo.createRecord` similar to posting. For example:

```
await agent.api.com.atproto.repo.createRecord({
  repo: myDid,
  collection: 'app.bsky.graph.follow',
  record: { subject: 'did:plc:xyz...abcdef', createdAt: new
Date().toISOString() }
});
```

The returned URI might look like `at://<your DID>/app.bsky.graph.follow/3kabc123...`. There is typically no additional content in a follow record aside from the DID of the followed user. To **unfollow**, you simply delete that follow record from your repo using

`com.atproto.repo.deleteRecord` (specifying the follow's collection and rkey). The Bluesky app and other clients will stop considering you as a follower of X once the record is gone.

Note: Follows are *public* records. Anyone can query who follows whom by listing follow records in repos. This is necessary in a decentralized network to let servers aggregate social graphs.

Likes and Reposts

"Likes" (upvotes) and "Reposts" (re-blogging someone's post) are also stored as records in the user's repo:

- A **Like** is an `app.bsky.feed.like` record. It contains a `subject` field pointing to the URI of the liked post (and usually the CID of that post for integrity) and a `createdAt` timestamp. Creating a like uses `com.atproto.repo.createRecord` on the `app.bsky.feed.like` collection. The SDK offers methods like `agent.like(postUri)` internally. To unlike, delete the like record.

- A **Repost** is an `app.bsky.feed.repost` record. It similarly has a `subject` (the URI of the post being shared) and `createdAt`. To repost, create this record; to undo, delete it. Reposts are essentially the "share/retweet" feature, and clients will show reposted content in feeds (often annotated as "User A reposted User B's post").

These records do not contain freeform text – they just reference the target content. They are public and can be fetched (e.g. there are endpoints to list who liked a post or who reposted it).

Record Management Endpoints Summary:

Endpoint	Method	Description and Usage
`com.atproto.repo.createRecord`	POST	**Create a new record** in a user's repo. Accepts JSON body with `repo` (DID or handle of the repo), `collection` (NSID of record type), and `record` (the record object) [15]. Returns the URI and CID of the created record [22]. Used for posts, likes, follows, etc.
`com.atproto.repo.putRecord`	POST	**Update or write a record at a specific key.** Similar to createRecord but used when the record's key is known or fixed (e.g. profile). Requires `rkey` field. Overwrites any existing record at that key.
`com.atproto.repo.deleteRecord`	POST	**Delete a record** from the repo, given `repo`, `collection`, `rkey`. After deletion, that record is removed from the user's data.

Endpoint	Method	Description and Usage
`com.atproto.repo.uploadBlob`	POST	**Upload a blob (media)** to the repo. Used for images or other media. Returns a `{ blob: { cid, mimeType, size, etc. } }` object to reference in records [19]. Blobs are stored separately (not in the repo MST) but are referenced by records.
`com.atproto.repo.getRecord`	GET	**Fetch a specific record** by its repo, collection, and rkey. For example, to fetch a given post or profile if you know its URI components. Returns the record JSON along with its CID.
`com.atproto.repo.listRecords`	GET	**List records** in a given collection of a repo (e.g. list all posts or all follows of a user). *Note:* This endpoint may be implemented under AppView (possibly as `app.bsky.feed.getAuthorFeed` or similar for posts).
`com.atproto.repo.applyWrites`	POST	**Batch transaction** of multiple writes. Allows creating, updating, and deleting multiple records in one atomic request (e.g. create several records at once or perform an update with a compare-and-swap on the repo's latest commit) [23]. This is advanced; it requires constructing a set of actions and an optional `swapCommit` (the last known repo commit CID for concurrency control).

Most higher-level actions correspond to creating or deleting records in one of the above collections. For example, blocking a user is technically adding an `app.bsky.graph.block` record (see Moderation section), and creating a list is an `app.bsky.graph.list` record, etc. The schema definitions in the lexicon tell you what fields each record contains.

Feed Reading and Generation

This section covers how to retrieve timeline feeds and how to produce custom feeds (an advanced feature introduced by Bluesky). Feed **reading** refers to fetching posts (your home feed, user feeds, thread view, etc.), while **feed generation** refers to creating algorithmic or curated feeds that others can follow.

Reading Feeds and Posts

- **Home Timeline:** The endpoint `app.bsky.feed.getTimeline` provides the authenticated user's home feed (timeline). This is typically a reverse-chronological view of posts from people you follow (the exact behavior could be influenced by user's feed preferences). It's a GET request that may accept parameters like `cursor` for pagination and `limit`. The result includes an array of feed posts (usually in a wrapped feed item format including posts and maybe reply/repost indicators). Example call:

```
const timeline = await agent.api.app.bsky.feed.getTimeline({ limit: 50 });
console.log(timeline.data.feed);  // array of posts and related info
```

Each feed post object contains the post record and often embedded viewer state (e.g. whether you've liked or reposted it).

- **Author's Feed:** To get all posts by a specific user, use `app.bsky.feed.getAuthorFeed`. This takes an `actor` parameter (DID or handle) and returns their posts in reverse chronological order. It's essentially that user's personal feed of their own content.

- **Fetching Specific Posts:** You can fetch specific posts by URI via `app.bsky.feed.getPosts` (which accepts a list of post URIs and returns the records plus embedded info like author profiles). To retrieve a post along with its reply thread context, use `app.bsky.feed.getPostThread`. That returns the post and the conversation thread (parents and replies) around it. This is useful for showing comments/replies.

- **Feed Metadata:** `app.bsky.feed.describeFeedGenerator` provides information about what feed generation services are configured on the server (e.g. the DID and endpoints of the feed generator service). This is more relevant in a federated context where custom feeds exist – essentially it helps clients discover where to query for custom feed content.

- **Likes and Reposts of a Post:** The endpoints `app.bsky.feed.getLikes` and `app.bsky.feed.getRepostedBy` (if implemented in AppView) allow listing which actors liked or reposted a particular post URI. These are useful for showing the "Liked by X and Y" or "Reposted by..." information. You supply the `uri` of the post and get back a list of actor profiles who have a corresponding like or repost record, plus pagination cursors.

- **Notification Feeds:** While not exactly "feed" in the timeline sense, note that `app.bsky.notification.listNotifications` exists to fetch your notification stream (mentions, follows, replies, etc.), and `app.bsky.feed.getTimeline` typically excludes your own notifications.

Pagination: Many feed endpoints (timeline, author feed, etc.) use cursor-based pagination. The response will include a `cursor` string when there are more items; pass this `cursor` in the next request to get the next page [24]. The client should not alter other query params when paginating.

Custom Feed Generation (Algorithmic Feeds)

One of Bluesky's distinctive features is support for **custom feeds** – alternative algorithms or curated feeds that users can subscribe to. A custom feed is backed by a Feed Generator service (an independent service that indexes content and picks posts for the feed). The feed is represented by a record of type `app.bsky.feed.generator` in some user's repo (often the service operator's repo), which other users can discover and follow.

Key points in feed generation:

- **Feed Generator Record:** This record (`app.bsky.feed.generator`) contains metadata like a URI or DID of the feed service, a display name and description for the feed, and an *algorithm identifier*. By creating this record, a developer "publishes" a feed. Others can then subscribe to it.

You might create such a record via `com.atproto.repo.createRecord` (likely with the feed service's DID and an identifier). *Only the record registration is on-chain (in your repo); the actual feed content comes from the feed service.* (In practice, the Bluesky app provides an interface for publishing feed generator records.)

- **Discovering Feeds:** Users can query `app.bsky.feed.getFeedGenerators` to get details of specified feed generators by URI (for instance, to fetch name/description and the author). There's also `app.bsky.feed.getSuggestedFeeds` which returns a list of feeds recommended (e.g. trending or featured feeds). These help users find feeds to follow.

- **Fetching a Custom Feed:** To fetch posts from a custom feed, clients use `app.bsky.feed.getFeed`. This takes a `feed` parameter (the at:// URI of the feed generator record, or its identifier) and returns a list of feed items. Under the hood, the AppView will proxy this request to the feed generator service defined by that record. The feed service responds with a **skeleton feed** – basically a list of post URIs (and maybe CIDs) in the order of the feed. The AppView then "hydrates" those URIs into full posts before delivering to the client [25] [26]. Thus, from the client perspective, `getFeed` works similarly to `getTimeline` (you get a list of posts), but the contents are determined by a custom algorithm.

- **Feed Generator Service API:** If you're implementing a feed generator service, you need to implement the XRPC method `app.bsky.feed.getFeedSkeleton` on your server. When the Bluesky app calls `getFeed` (as above), the user's AppView will call the feed generator's `getFeedSkeleton` endpoint (using the service DID in the `service` header). The feed generator receives query params like `feed` (which feed to serve, in case one service hosts multiple feeds) and pagination params, then should respond with an ordered list of feed item URIs (and cursors). The AppView will then fetch those posts from the network and return them to the user. Essentially, the custom feed service's job is to maintain an index of recent posts (likely by consuming the Firehose, see below) and to score/select posts for the feed.

How to Build a Feed Generator: In short, building a custom feed involves: (1) **Ingesting posts** – typically by subscribing to the global firehose (`com.atproto.sync.subscribeRepos`) to get all new posts in real-time, then filtering or storing those you need, and (2) **Implementing the feed query** – i.e., deciding which posts (from your database) to return for the feed when asked.

Bluesky's guidance for feed generation is: *"For most use cases, we recommend subscribing to the firehose at* `com.atproto.sync.subscribeRepos`*. This WebSocket will send you every record that is published on the network… Feed Generators do not need to provide hydrated posts, you can index as much or as little of the firehose as necessary."* [27]. In practice, your feed service might only keep recent posts (e.g. last 48 hours) in memory or a database, and when `getFeedSkeleton` is called, apply your algorithm to pick the top N posts to return [27] [28].

Firehose (Global Feed Stream): `com.atproto.sync.subscribeRepos` is the event stream that emits *all* repository updates (across all users on a server, or on an aggregator for the whole network). It's how a feed service can get every new post, like, follow, etc. The feed generator can ignore records it doesn't care about (e.g. only index `app.bsky.feed.post` records for a specific feed about posts, or maybe also `app.bsky.graph.follow` if needed). The Firehose stream returns a sequence of **commit events** with metadata and a list of record operations (creates, updates, deletes) included in each commit [29]. Each event has a `seq` number and the stream supports resumable consumption via cursors. No auth is required for `subscribeRepos` (it's a public feed of public data).

Using the official TypeScript library, you can consume the firehose like so:

```
import { AtpAgent, AtpSessionEvent, AppBskyFeedPost } from '@atproto/api';

const agent = new AtpAgent({ service: 'https://bsky.social' });
for await (const ev of agent.subscribeRepos()) {
  if (ev.repo) {
    for (const op of ev.operations) {
      if (op.action === 'create' && op.collection === 'app.bsky.feed.post') {
        const postRecord = op.record as AppBskyFeedPost.Record;
        console.log(`${op.repo} posted: ${postRecord.text}`);
        // apply your filtering/storage logic here
      }
    }
  }
}
```

(Above is a simplified illustration – the actual firehose API may provide higher-level utilities. The Bluesky team's example shows filtering and processing in the `FirehoseSubscription` class [30].)

Feed Generation Endpoints Summary:

Endpoint	Description
`app.bsky.feed.getTimeline` **GET**	Get the authenticated user's home timeline (posts from followed accounts, etc.). Supports `cursor` pagination [31].
`app.bsky.feed.getAuthorFeed` **GET**	Get a specific actor's feed (their posts). Params: `actor` (DID or handle), pagination.
`app.bsky.feed.getPosts` **GET**	Fetch specific posts by URI (batch). Useful to hydrate a set of post URIs.
`app.bsky.feed.getPostThread` **GET**	Retrieve a post and the surrounding thread (parents and replies). Param: `uri` of the post, plus options for depth, etc.
`app.bsky.feed.getLikes` **GET**	List who liked a given post. Param: `uri` of the post. Returns an array of actors (with small profile info) and their like timestamps.
`app.bsky.feed.getRepostedBy` **GET**	List who reposted a given post. Param: `uri` of the post. Similar response to getLikes.
`app.bsky.feed.getSuggestedFeeds` **GET**	Get a list of suggested feed generators for the user (e.g. trending feeds). Returns feeds (with their record info).
`app.bsky.feed.getFeedGenerators` **GET**	Fetch metadata about feed generator records by their URIs or DIDs. Can be used to look up feed info (like name, description) in bulk.

Endpoint	Description
`app.bsky.feed.getFeed` **GET**	Fetch the content of a feed from a feed generator. Param: `feed` (at:// URI or identifier of the feed). Returns a feed of posts (skeleton is hydrated by server).
`app.bsky.feed.describeFeedGenerator` **GET**	Provides information about the feed generator setup on the server (e.g. the DID of the feed service, and possibly supported feed NSIDs). This helps clients route feed queries.
Feed Generator Service Endpoints:	*(Not part of the public API, but for custom feed implementers)* You must implement `app.bsky.feed.getFeedSkeleton` (and any relevant lexicons like `app.bsky.feed.describeFeedGenerator` on your feed server if needed). This query should accept feed identifiers and return a list of items (post URIs) with cursors. The Lexicon for skeleton feeds ensures the AppView can parse it and perform hydration [25] [27].

With these, a user or client can seamlessly switch between different feeds. Custom feeds are a powerful extension point – they run on independent services but use the AT Protocol to integrate with the network.

Moderation Tools

Bluesky provides a few layers of moderation controls via the AT Protocol APIs: user-level moderation (blocking and muting other users, which affects your view), reporting content to server moderators, and administrative moderation (for moderators to take action). This section focuses on the user-level and reporting aspects, since admin tools are generally not open to third-party apps without special roles.

Blocks (Account Blocking)

A **block** is a stronger action than unfollow – it prevents another account from interacting with you or seeing your content in their feed (and vice versa, on compliant servers/clients). In AT Protocol, blocks are implemented as *public records* in the user's repo, similarly to follows [32]. The record type is `app.bsky.graph.block`, and it's nearly identical in schema to a follow: it contains a `subject` (the DID of the account being blocked) and a `createdAt` timestamp [32]. When you block someone via the client, the app creates a `graph.block` record in your repo.

Public vs Private: It may sound surprising, but block records are public (anyone could see that A has blocked B by looking at A's repo) [33] [34]. This is a deliberate design choice: in a decentralized network, servers need to know your blocks to enforce them (e.g. filtering content). All compliant servers and clients will use these records to prevent blocked users from interacting (no replies, mentions, follows, etc., and profiles may appear as blocked) [35]. However, nothing in a blocked user's own data is deleted – the enforcement is done by other participants (PDSes, app views, clients) who refuse to relay or display content that violates a block [36] [37]. In practice, Bluesky's app UI also prevents you from seeing the blocked account's content unless you log out.

To **create a block**, add the `app.bsky.graph.block` record via `createRecord` (the client SDK provides a method `agent.block()` which does this). To **unblock**, delete that record. For example, in TypeScript:

```
await agent.api.com.atproto.repo.createRecord({
  repo: myDid,
  collection: 'app.bsky.graph.block',
  record: { subject: targetDid, createdAt: new Date().toISOString() }
});
// … later, to unblock:
await agent.api.com.atproto.repo.deleteRecord({
  repo: myDid,
  collection: 'app.bsky.graph.block',
  rkey: '<rkey-of-block-record>'
});
```

You can retrieve your block list by querying your repo for `graph.block` records (there isn't a high-level endpoint for "list my blocks" in AppView yet, but one can use `listRecords` or the firehose to see them). The Bluesky app does show users a list of whom they've blocked [33].

Effects of Blocking: The protocol and clients ensure that a blocked user cannot follow you, see your posts in their home feed, reply/mention you, or otherwise interact [38] [36]. They may still see your *public* content by other means (logged out, alternate account) since the data is public, but *direct* interaction is barred. Block relationships are public as a necessary trade-off to allow network-wide enforcement in a federated system [34] [39].

Mutes (Hiding Content)

A **mute** is a personal setting to hide an account's posts from *your* view, without alerting the other person and without preventing their interactions elsewhere. Mutes are not represented as public records (because they don't need to be enforced by others). Instead, the protocol offers endpoints to mute/unmute actors and these preferences are stored privately (likely on your PDS). The endpoints are:

- `app.bsky.graph.muteActor` – a **procedure (POST)** to mute a given actor (user). You provide the DID or handle of the actor to mute. After muting, their posts won't show in your timeline and you won't get notifications from them. This call requires authentication and is scoped to your account's PDS [40].
- `app.bsky.graph.unmuteActor` – POST to remove a mute on an actor.

Muting is client-specific in effect (each user's client should respect their mute list). Since mutes don't produce a record in your repo, they're not visible to others. Think of it as an entry in your account's "muted users" list on the server.

There are also "mute lists" features (e.g. muting all posts with a certain label or from a certain list of users), but those are part of content filtering and lists, discussed below.

Reporting Content (Moderation Reports)

If a user encounters spam or abuse, they can **report** it to the service moderators. The endpoint for this is `com.atproto.moderation.createReport`. This is an authenticated procedure that *submits a*

moderation report regarding an account or record [41]. You specify the subject of the report (either an account DID or a specific record URI) and a reason/reasonType (such as spam, harassment, etc., based on server-supported categories). The report is sent to the user's PDS, which will typically forward it to the appropriate moderation service (Bluesky runs a centralized moderation service called Ozone).

Key points: - `createReport` requires authentication and is implemented by the moderation service (with the PDS proxying the request) [41]. This means as a client developer, you just call the endpoint on the user's PDS; you don't need to know the internals of Ozone. - The request body includes fields like `subject` (with `$type` specifying whether it's a user or a record), `reasonType` (an enum of predefined reasons like `com.atproto.moderation.defs#reasonSpam`, `#reasonHarassment`, etc.), and an optional free-text `reason` description. - The response will confirm the report was received (often returning an ID for the report). What happens next (e.g. moderator reviewing it) is outside the scope of the client API.

There are currently no public endpoints to query the status of your reports via the client API.

Other Moderation Tools

- **Account Takedown/Suspension:** In cases of severe violations, moderators can label accounts with special labels that cause content to be taken down or accounts suspended (see Labels section for `!takedown`, `!suspend`). Regular client apps won't have access to admin endpoints (like `com.atproto.admin.disableAccount` or similar); those require moderator privileges and are not covered here.

- **Invite Codes / Account Management:** Bluesky's early access is invite-based. The endpoints for creating or consuming invite codes (`com.atproto.server.createInviteCode`, etc.) exist but are typically restricted. They aren't central to general app development, so we omit them here.

- **Application Permissions:** The AT Protocol also defines a scheme for app-specific permission scopes and background moderation through *"permission scopes"* and *"labelers"*. For example, an app might request permission to post on your behalf or to read your follows. This is tied into OAuth and the lexicon `com.atproto.repo.applyWrites` (the `writes` might carry an `authority` field). This is advanced usage; standard clients using the official SDK will handle the needed scopes when using OAuth.

Moderation Endpoints Summary:

Endpoint	Description
`app.bsky.graph.muteActor` **POST**	Mute an actor (user) for the authenticated user [40]. After muting, that actor's content is hidden from this user.
`app.bsky.graph.unmuteActor` **POST**	Unmute a previously muted actor.
`com.atproto.moderation.createReport` **POST**	File a moderation report about an account or record [41]. The report goes to the server's moderators. Requires auth.

Endpoint	Description
(`app.bsky.graph.block` *record)*	*Not a separate endpoint – block is done by creating/deleting this record.* [32] The presence of the record indicates a block.
`app.bsky.graph.getBlocks` (potential)	**(If implemented)** Get the list of accounts the user has blocked. Currently, one may need to use sync APIs to enumerate `graph.block` records.
`com.atproto.admin.*` (various)	Administrative moderation actions (e.g. take down content, disable account). These are **not** accessible to third-party apps unless the app is an admin tool. They require higher privileges.

Labels and Content Filtering

Bluesky uses a system of **content labels** to categorize and filter content. Labels are pieces of metadata attached to records or accounts indicating things like "nsfw", "spam", "politics", etc., which can be used by clients to decide what to show or hide. The AT Protocol defines a flexible, composable labeling system and a way for labels to be distributed.

Label Basics

A **Label** is essentially a small record with: a **source DID** (who created the label), a **subject** (what is being labeled – could be a record URI or an actor DID), a **value** (the label term, e.g. `nsfw` or `politics`), and metadata like `createdAt` and optional `neg`ation or expiration [42] [43]. Labels are **separate** from the records they describe; they aren't fields inside posts, but rather stand-alone assertions that "Subject X has label Y". For example, a moderation service might label a particular post as `porn` or an account as `spam`.

Some labels can also be **embedded** in records by the author (self-labeling). For instance, a user might self-label their post as `nsfw` by including a label in the record (as per lexicon, some record types allow a `labels` field). But generally, labels are created by **labeler services** (automated or human moderation systems).

Important details from the spec: - Labels exist as **self-authenticating objects** (they can be signed by their creator). The canonical form is binary (CBOR) for signing, but in JSON they appear with similar fields [44] [43]. - A label's `val` (value) is typically a short ASCII string (like a tag). There is intentionally no strict global ontology defined in the base spec – values are open-ended, but ecosystem conventions are emerging (e.g. `nsfw`, `self-harm`, `spoiler`, etc.). Certain prefixes like `!` are used for "behaviors" (e.g. `!takedown` indicating content removed) [45] [46]. - Labels can be **negated** by setting `neg: true` which means the label is revoking a previous label of the same value [47] (e.g. a post was mislabeled and later the labeler wants to withdraw that label).

Label Distribution and Queries

Labels are typically produced by a **moderation/labeler service** (like the Bluesky Ozone service) and need to be distributed to PDSes and app views so that content can be filtered appropriately. The ATProto spec defines two primary endpoints for this:

- `com.atproto.label.subscribeLabels`: a **WebSocket event stream** clients (or services) can subscribe to, in order to receive *new labels in real-time*. It works like other XRPC subscription endpoints – you connect to `/xrpc/com.atproto.label.subscribeLabels` and receive a continuous stream of label events. It supports *backfill*: you can specify a cursor (sequence number) to get historical labels, even all the way back to 0 to ingest the full label history [48]. This stream is similar to the repo "firehose" but for labels specifically. When a label is negated (redacted), the stream will include an event indicating that (often by still sending the negation as a new label with `neg:true`) [49].

- `com.atproto.label.queryLabels`: a **HTTP GET** query endpoint for looking up labels by various filters. You can use it to find all labels on a specific subject (e.g. get all labels on a particular record or account), or do a broad query of recent labels, with pagination [50]. This is useful if you want to check, for example, what labels exist for a given post (maybe to display warning UI or blur it). It's also the fallback for clients that can't maintain a live subscription.

These endpoints are often implemented by the **labeler service** (not necessarily by every PDS). For Bluesky, the Ozone moderation service likely hosts these. A PDS can proxy these requests (similar to how other service endpoints are proxied).

Availability: Note that not all labeler services might expose these publicly – the spec allows that label data might be access-controlled [51]. Bluesky's approach has been to have a centralized moderation service that does make labels available to app views with appropriate trust.

Using Labels for Filtering

Clients (and app views) use labels to filter content according to user preferences. For example, a post labeled `nsfw` might be hidden behind a content warning or not shown at all if the user's settings disallow NSFW content.

Bluesky defines **preference records** that capture a user's content filtering choices. These are accessible via `app.bsky.actor.getPreferences` and settable via `app.bsky.actor.putPreferences`. Preferences include things like `adultContentPref` (a boolean pref for viewing NSFW content) and **label-specific preferences** (e.g. you could have preferences for "show/hide content with label X") [52] [53]. The lexicon `app.bsky.actor.defs#contentLabelPref` likely defines a structure for preferences on specific label values. For instance, a user might say "Blur content with `violence` label" or "Hide content with `porn` label".

When you fetch a feed or a profile, the server (AppView) will by default apply some filtering: it might exclude content with very severe labels (like illegal content) entirely, and for other labels, it might include a label metadata in the response so the client can decide to blur or hide. The client can also request specific handling via HTTP headers:

- `Atproto-Accept-Labelers` **Request Header:** A client can send this header to indicate which labeler DIDs it trusts for labels and whether it wants auto-redaction. For example: `Atproto-`

`Accept-Labelers: did:example:mod;redact, did:plc:abcdefg` [54] [55]. In this header, listing a DID with `;redact` means "I want content labeled with the special labels `!takedown` or `!suspend` by this labeler to be fully removed (redacted) from results" [55]. Without `redact`, the client might just receive the labels and decide locally what to do. If the header is not provided, servers may use a default set of labelers and defaults for redaction [56].

- `Atproto-Content-Labelers` **Response Header:** The server responds with this header to indicate which labeler sources were applied in filtering the content [54]. It essentially echoes which labelers' data was used. This helps clients know if their requested label filters were honored.

In simpler terms: by using these headers, an app can control whether the server should proactively remove content (like illegal stuff) versus just send labels. Typically, a client might let the server handle the heavy removals by using `redact`, and then handle milder content warnings itself (blurring images, etc., based on labels included in the response).

Example Flow: Suppose a post is labeled as `nsfw` by a moderation service `did:plc:mod123`. If the user's `adultContentPref` is "hide", the app might either tell the server to not return such content or it will get the content but with a label attached indicating NSFW. The client could then decide to not display it unless clicked. In the response JSON for feed posts, there is usually a `labels` field listing any labels on that post (with their source and value). The client can read that and apply the appropriate UI (blur image, collapse text, etc.).

Creating Labels

While most labels will come from automated services or moderators, developers could also use the labeling system for other purposes. The ATProto lexicon defines `com.atproto.label.applyLabels` (not certain if exposed to clients) which would allow writing labels (e.g. if you build a custom moderation bot). However, in general third-party app developers won't be arbitrarily tagging others' content with labels unless they run their own moderation service and users opt into it.

Label Summary

- Labels are **metadata tags** on content/accounts, used primarily for moderation.
- They are distributed via `subscribeLabels` (real-time stream of labels) and `queryLabels` (on-demand lookup) [57].
- Clients should respect labels by filtering content according to user settings. Bluesky's reference app uses user prefs to determine if content with a given label is shown, blurred, or hidden.
- Special label values like `!takedown` or `!suspend` are used by servers to indicate content or accounts that have been removed from the network (e.g. due to ToS violations). If a post is labeled `!takedown` by a trusted labeler, a compliant app view will *redact* it entirely (not just hide – it won't even appear in results) [55]. The `redact` flag in the Accept-Labelers header instructs whether to do this automatically.
- Labels are versioned objects (current version = 1) and can be cryptographically signed [44]. This allows trust: you might trust labels from the did:plc:mod (Bluesky's mod team) and not from unknown sources.

Label/Filtering Endpoints:

Endpoint	Description
`com.atproto.label.subscribeLabels` (WS)	Subscribe to stream of all new labels (with backfill). Used by services or clients that need live label updates [49].
`com.atproto.label.queryLabels` **GET**	Query labels by criteria. Can filter by subject URI, by source, by value, etc., and paginate [50].
`app.bsky.actor.getPreferences` **GET**	Get the user's preferences, which include content filtering settings (e.g. `adultContentPref`, label display prefs) and other app settings. The response will list an array of preferences. Each preference might have `$type` like `app.bsky.actor.defs#adultContentPref` with an `enabled` flag, or `#contentLabelPref` with a label and visibility setting.
`app.bsky.actor.putPreferences` **POST**	Update the user's preferences. The body requires a list of preference objects to set [53]. This is how a user (through a client) can change their content filter settings.
(Content labels in records)	(Many feed and profile fetching endpoints will include a `labels` field in their response objects, e.g. each post may have an attached list of labels applied to it. This is not a separate call, but part of the data model you should be aware of when rendering content.)

In summary, to implement content filtering in your app: fetch the user's prefs (`getPreferences`), include appropriate `Atproto-Accept-Labelers` headers on feed/profile requests (if you want server-side redaction for extreme content), examine the `labels` on content in responses, and enforce the user's choices (hide/blur content with those labels). The label system is powerful and evolving, but official sources encourage using it for a safer user experience [58] [55].

Lexicons and Schema Usage

The AT Protocol uses a schema system called **Lexicon** to define all APIs (procedures, queries) and data structures (records, objects). Understanding lexicons is key to using the APIs effectively, especially if you plan to define your own record types or work at a low level.

What is a Lexicon? A lexicon is a JSON schema (somewhat similar to JSON Schema spec) with AT Protocol-specific features. Each lexicon has a **NSID** (Name-space ID, often reverse-DNS format) as its identifier. For example: - `com.atproto.repo.createRecord` is a lexicon ID for an XRPC procedure. - `app.bsky.feed.post` is a lexicon ID for a record type.

Lexicons describe the types (data models) and the **RPC methods** (endpoints). There are different lexicon "primitive" types: - **Record** – defines a record object that can live in a repo (with fields, constraints, etc). - **Query** – defines a GET endpoint (no side effects, primarily for retrieval). - **Procedure** – defines a POST endpoint (can have side effects/changes). - **Subscription** – defines a continuous feed (WebSocket subscription).

Each lexicon JSON file can contain multiple definitions (often a "main" definition plus perhaps sub-schemas or union types). The Bluesky team maintains an official repository of lexicon definitions (`bluesky-social/atproto` on GitHub, under the `lexicons/` directory).

Using Lexicons in Practice: Most developers will rely on generated code or the official SDK rather than manually parsing lexicon JSON. For instance: - The `@atproto/api` TypeScript library includes TypeScript types and methods for all official lexicons (e.g. it has an interface for `AppBskyFeedPost.Record` which matches the schema of a post, so you get type-checking in code). - There is a CLI tool `lex` (and an NPM package `@atproto/lex`) that can generate TypeScript classes from lexicon JSON. This is used when you create a *custom schema* for your own record type. For example, the "Status" example in the ATProto docs defines a new schema `xyz.statusphere.status` and then runs `lex gen-server` to produce TS interfaces and validation code for that schema [59] [60]. - The AT Protocol SDKs in other languages similarly incorporate lexicon specs (for example, atproto.py, atproto.dart, etc., have models generated from lexicons).

When interacting with the API, the lexicon ID determines the HTTP path (after `/xrpc/`) and the expected request and response shapes [61]. For example, if you call `/xrpc/com.atproto.repo.createRecord`, the server knows to expect JSON matching the `createRecord` schema (with `repo`, `collection`, etc.), and it will return data per that schema (URI & CID). If something is wrong (missing field, type mismatch), the server can validate against the lexicon and return an error.

Lexicon Versions: Currently lexicon version is indicated by a top-level `"lexicon": 1` field in the JSON schema (version 1). As the protocol evolves, schemas might get new definitions or revisions, but they try to add non-breaking changes or bump version if necessary. Your client should ideally be generated or updated to match the lexicons of the server it targets. The official Bluesky AppView's lexicons are documented in an OpenAPI-like format at docs.bsky.app, but the truth is in the lexicon JSON files.

Fetching Lexicon Schemas: There is an XRPC method `com.atproto.lexicon.resolveLexicon` that allows you to fetch a lexicon by NSID at runtime [62]. This can be useful if you want to introspect the schema or ensure you have the latest. However, most clients will have the schema predefined for known IDs.

Schema Composition: Lexicons allow references to other schemas. For example, `app.bsky.feed.post` record uses `#replyRef` or refers to `app.bsky.feed.embed.*` schemas for the embed field. Understanding the IDs (like `record$ref`) and the way they link is important if working at the schema level. If you use the official libraries, much of this is handled for you (e.g. you get a `FeedPost` object with possibly an `embed` property typed as union of known embed types).

Defining Custom Schemas: Developers can define new record types for their applications (as seen in the "statusphere" example) [21] [59]. Key steps: 1. Choose a unique **NSID** under a domain you control (e.g. `io.myapp.something` or `xyz.myname.recordType`). This prevents collisions and signals ownership. 2. Write a lexicon JSON with the structure. For records, you typically specify `"type": "record"`, and include fields in a `"properties"` object with types (string, integer, boolean, etc.). You also define how the record's key is generated: either `"key": "tid"` for an auto-generated TID or `"key": "literal"` or a specific fixed key. 3. (Optional) Provide descriptions and constraints (minLength, maxLength, formats, etc.). The lexicon language supports many JSON Schema keywords (with some differences). 4. If your schema is a new XRPC method (query or procedure), define the parameters, input, and output fields accordingly. 5. Use the `lex` tool to generate code, or manually integrate the schema into your app's type system. In the TS example, after generating, they got a

`Status.isRecord()` and `Status.validateRecord()` function to ensure an object conforms to the schema before posting it [63] [64]. 6. Use repository APIs to *create records of your new type*, and potentially write clients to read them. Other clients won't know about your schema unless they also load it – but they will still store and sync it (unknown record types won't break the network, they are just treated opaquely if the client doesn't understand them).

In summary, lexicons make the AT Protocol **self-describing and strongly typed**. By following schemas, clients ensure they send properly formatted data and can expect certain response shapes. Always refer to official lexicon definitions for the exact fields and types expected for each endpoint or record. The Bluesky docs site and the `atproto/lexicons` repo are authoritative sources for these definitions [21].

Advanced Topics

Finally, we cover some advanced or less common aspects of the AT Protocol API: the Firehose event stream, DID resolution and handles, repository synchronization, and the new direct messaging (Chat) APIs.

Firehose API (Repo Subscription Stream)

The **Firehose** refers to the real-time feed of **all** repository events in the network (or on a given server). We touched on this in the feed generation section: `com.atproto.sync.subscribeRepos` is the primary endpoint for this. It's a WebSocket subscription that sends a continuous sequence of events representing repo updates (commits). Each event can include multiple record operations (create/update/delete). The firehose is used by services that need to maintain an index of everything (e.g. search index, custom feed generator, data analytics, etc.).

Important details: - **No Auth Required:** By default, anyone can subscribe to the global feed of public data. On the main Bluesky ecosystem, `bsky.social` exposes `subscribeRepos` for at least the domains it knows about. In a federated future, one might subscribe to multiple servers or a relay that aggregates them. - **Event Structure:** The events are defined in lexicon `com.atproto.sync.subscribeRepos#commit` (and also `#handle` for handle changes, `#info` for heartbeat/info messages, etc.). A `#commit` event includes the `repo` DID, a `seq` number, a `commit` CID, an optional `prev` (prev commit), and crucially an `ops` list detailing each record change [65] [29]. Each op has a `action` (create/update/delete), `collection`, `rkey`, and possibly the `record` value for creates/updates or the `cid` being deleted. There's also a `tooBig` flag if a commit had too many changes to send inline, in which case you'd need to fetch the remainder via sync (the spec advises commits should generally be small) [66] [67]. - **Backpressure & Resume:** The subscription allows a `cursor` query param to resume from a certain sequence. If your client disconnects, you can reconnect with `?cursor=<last seen seq>` to continue. Typically, a firehose client should process events sequentially and keep track of the last seq processed. - **Validation:** The protocol expects consuming services (like relays or app views) to validate the events – e.g. verify signatures on commits, ensure the DID's data matches the origin, ignore events for users who have moved servers, etc. [68] [69]. If you're building something like a relay or full indexer, consult the "Firehose Validation Best Practices" [70] [69]. For a simpler consumer (like just populating a feed or logging posts), you might not implement all validation, but be aware of potential anomalies. - **Use Cases:** Custom feeds (as discussed), **analytics** (e.g. measuring activity), **backup** (one could record all events to reconstruct state), or building a **search engine** or **aggregator** that needs all data.

To consume `subscribeRepos` in TypeScript, the official SDK provides a high-level interface (`agent.subscribeRepos()` returns an async iterable or event emitter of events). In other languages, similar support exists (e.g. atproto.py's Client has subscribe_repos method).

DID Resolution and Handle System

AT Protocol separates **identity** (DIDs and handles) from servers. Each user has a stable DID (e.g. `did:plc:abcd1234` – PLC stands for Personal Data Locker, a specific blockchain for DIDs, or could be `did:web:username.com`). They also have a human-readable **handle** (like `alice.bsky.social` or a domain name they own). The mapping between handle and DID can change if a user moves to a different server, but the DID remains the primary key for identity and is what's used in all records and API calls [71] [72].

Resolving a Handle to a DID: To get the DID for a handle, use `com.atproto.identity.resolveHandle`. For example, resolving `"alice.bsky.social"` might return `did:plc:1234567...` [73]. This endpoint is useful because in some contexts (like browser-based apps) you can't do DNS lookups directly, so the PDS offers an API call to do it [74]. Under the hood, resolution follows the rules in the Handle spec: - First, check DNS: look for a TXT record at `_atproto.<handle>` with `did=<DID>` [75]. - If not found or handle is too long for DNS, fallback to an HTTPS fetch at `https://<handle>/.well-known/atproto-did` which should contain the DID [76]. - Then confirm that the DID Document for that DID contains the handle (to avoid spoofing) [77] [78].

The `resolveHandle` endpoint automates this and returns the DID quickly (it might not do the full bidirectional check for speed, but generally the expectation is the handle is valid or the DID Document would be updated accordingly) [79].

DID Document and Service Discovery: Once you have a DID (like `did:plc:abc...`), the client or PDS can resolve the DID document (via the PLC directory or DID method). The DID Document will tell you the DID's *public key* (for verifying signatures) and also the URL of the user's PDS (the service endpoint) [80]. For `did:plc` (Bluesky's own DID method), there's a PLC registry on a blockchain; for `did:web:example.com`, the DID document is basically hosted at `https://example.com/.well-known/did.json`. The ATProto Identity spec covers this in detail [71] [81].

So, if a user enters "alice.bsky.social" into your app: 1. Call `resolveHandle(handle)` → returns `did:plc:xyz`. 2. Then maybe call `com.atproto.identity.resolveIdentity` (if available) or directly use the DID to find their PDS. In many cases, your `BskyAgent` can be initialized with just the handle and password; it will internally resolve the DID and PDS. 3. All subsequent requests for that user's data (especially writes) should go to the PDS from the DID Document (for `bsky.social` users, that's `https://bsky.social` itself).

Handle updates: If a user changes their handle (e.g. they bought a domain and want to use that), the DID stays the same but now resolves to a new handle. The protocol emits handle change events via `subscribeRepos` (a `handle` event type). Also, `app.bsky.actor.searchActors` and similar endpoints can help find users by handle or display name.

DIDs in URIs: All AT Protocol repository URIs use the DID, not the handle: e.g. `at://did:plc:abcd123/app.bsky.feed.post/abcdef123` identifies a post in that DID's repo. This ensures the link remains valid even if the user changes handle or moves servers (as long as the DID resolution is updated to the new server).

For convenience, some endpoints accept either DID or handle as an `actor` parameter. The system will resolve if needed (for example, `app.bsky.actor.getProfile?actor=alice.bsky.social` works with a handle).

Repository Sync and Backup

Beyond the firehose, there are specific **sync** endpoints to retrieve repository data in bulk or by shards: - `com.atproto.sync.getRepo` – download the entire repo state of a user as a CAR file (Content-addressable archive). This can be used for backup or for migrating an account to a new server. It returns a stream of blocks (commits and records). You can specify a `since` commit to get an incremental diff, or get the full history. - `com.atproto.sync.getRecord` – (distinct from repo.getRecord) fetch a specific record's block by its URI. Useful if you want the raw CBOR of a record. - `com.atproto.sync.getLatestCommit` – quickly fetch the latest commit CID and revision of a repo (to see if you're up-to-date). - `com.atproto.sync.listBlobs` / `getBlob` – list and fetch blobs from a repo (e.g. images the user has uploaded). - `com.atproto.sync.listRepos` – list all repos known to the server, possibly with info like last update. On Bluesky's AppView, this might list all user DIDs and their info – likely an admin or federation tool. - `com.atproto.sync.notifyOfUpdate` – a way for a PDS to hint to another service that a repo was updated (like a lightweight webhook). - `com.atproto.sync.requestCrawl` – ask a crawler service to crawl a repo (likely for federation, when a server learns about a new user or after downtime). - `com.atproto.sync.subscribeRepos` – (the firehose we discussed).

These endpoints support building a full data replication mechanism. If you run a PDS or an indexing node, you'd use them to keep mirrored copies of user data. For app developers, these are usually not needed unless you're doing something like exporting user data or building an app view from scratch.

Direct Messages (Chats)

Note: Direct messaging (DMs) on Bluesky was introduced in 2024 and is still evolving. As of now, DMs – also referred to as "Chats" – are one-to-one private conversations. They are not stored in your public repo; instead, Bluesky uses a central service for handling DMs.

Architecture: DMs use a separate lexicon namespace: `chat.bsky.*`. The reason is that DMs likely involve encryption and privacy considerations that don't fit the public repo model. Bluesky's approach is to route chat messages through a **central chat service** (with DID `did:web:api.bsky.chat`), while still using ATProto XRPC calls to interact with it [82]. The user's PDS acts as a proxy to this service.

Key endpoints in the Chat API (as of now): - `chat.bsky.convo.create` – possibly to create a new conversation (or this might happen implicitly on first message). - `chat.bsky.convo.sendMessage` – send a message in a conversation (or start a new convo) [83]. - `chat.bsky.convo.listConversations` – list your DM conversation threads. - `chat.bsky.convo.listMessages` – get messages in a conversation (with pagination). - `chat.bsky.convo.muteConvo` / `unmuteConvo` – mute notifications from a particular conversation. - There might also be endpoints for things like adding a member (if group chats become a thing) but currently DMs are likely only 1:1.

Using the Chat API: From a client perspective, to send a DM: 1. **Authentication:** You must be logged in (obviously, since it's user-specific). 2. **Locate the conversation:** If you have an existing conversation ID (maybe the DID of the other user or a conversation identifier), you use that. If not, the first message might create the convo. 3. **Call sendMessage:** e.g.

```
await agent.api.chat.bsky.convo.sendMessage({
  conversationId: 'abcdef123' , // or possibly { recipients: [did1, did2] }
if starting new
  text: 'Hi there!',
  createdAt: new Date().toISOString()
});
```

(The exact parameters depend on lexicon; conversation might be identified by the other user's DID if strictly 1-to-1.)

The message will be delivered to the chat service, which will store it and deliver to the other user's PDS for their client to fetch.

1. **Receiving messages:** Likely via `chat.bsky.convo.listConversations` (to get all threads and see unread counts) and `chat.bsky.convo.listMessages?conversationId=X` to get the messages. There may also be a subscription endpoint for chats (possibly not yet, but maybe in the future to stream new DMs).

Important: Bluesky's DMs are currently not E2E encrypted (as far as known at launch; they plan to add encryption later). For now, the messages are visible to the chat service (Bluesky's server) – so they are private from other users, but not from the platform operators. Keep that in mind if building or using DMs.

From the API perspective, the main quirk is the **service proxying**: The `sendMessage` call must be directed to the user's PDS, but with a header indicating to forward it to the chat service. The official docs note that the request should set the service DID header to `did:web:api.bsky.chat` [82]. The `BskyAgent` likely handles this automatically when you call the chat methods.

Conversations and Message Identifiers: Each DM conversation likely has an ID (maybe a DID or a composite of the participants). The lexicon might define a "conversation" record or object. The first message might cause a conversation object to be created (with a TID) that both participants reference. The details can be found in the `chat.bsky.convo` lexicons, which the official SDK should abstract away.

Moderation for DMs: The system does allow reporting DMs to mods (the Safety team account mentioned you can report DMs). Under the hood, that might mean there are internal hooks for moderators to inspect reported DMs (perhaps DMs are stored in a way that mods can decrypt or fetch given a report). For client dev, if implementing a DM interface, consider offering "Report" options – possibly by calling the same `createReport` endpoint on the message URI (if DMs have URIs) or on the user with a reason indicating it's about a DM.

Federation and DID Switching

As advanced topics: - **DID Rotation:** Users can rotate the signing key in their DID document. This is mostly handled by the PLC system and PDS (the repo updates need new signature, etc.). Not directly an API call except through the identity system (perhaps `com.atproto.server.rotateSigningKey` exists for a user to do this). - **Account Migration:** Users may move from one PDS to another. There is an "account migration" guide and presumably an endpoint to initiate migration. Possibly `com.atproto.account.requestTransfer` or similar. This is still experimental and might not be fully

automated in current APIs. - **Federation Status:** Endpoints like `com.atproto.sync.getRepoStatus` or `sync.listReposByCollection` can be used by servers to coordinate data. For example, if you run your own PDS, you might use these to ensure you have all data from others.

These are beyond normal client scope but good to be aware of: the AT Protocol is built for open federation even though Bluesky is currently running in a mostly centralized mode.

Error Handling and Best Practices

To make your integration robust, keep in mind the following:

Standard Error Format: All XRPC errors come in a standard JSON shape with an `"error"` field (a short string code) and a `"message"` field (human-readable description) [84]. For example, a request with missing parameters might return: `{ error: "InvalidRequest", message: "missing parameter 'repo'" }` (HTTP status 400). Always check for non-200 HTTP responses and parse the JSON to get these fields. The error codes are defined per lexicon method (each method can specify error names). For instance, `authRequired` might be an error for endpoints needing login (HTTP 401), `InvalidToken` for expired JWT, etc. Use these codes to handle specific cases (like prompting re-login on auth errors).

Rate Limiting: The API may enforce rate limits. An HTTP 429 error code indicates too many requests. The response might include `Retry-After` header (if not, you should implement an exponential backoff). Best practice is to avoid hammering the API with aggressive polling; instead use the provided streams (timeline, firehose, etc.) or reasonable intervals.

Data Consistency: When writing data (creating or deleting records), there is a chance of race conditions if multiple writes happen quickly. The AT Protocol repository has an **optimistic concurrency** mechanism via *compare-and-swap*. The `createRecord`, `putRecord`, and `deleteRecord` endpoints accept optional `swapCommit` and/or `swapRecord` fields (depending on method) [85]. `swapCommit` is a CID of the repo's current head that you expect; if the repo head has advanced (someone else wrote something) the operation will fail with a `ConcurrentWriteFailed` error, telling you to refetch the latest state. This is analogous to an ETag precondition. If you build an app with frequent concurrent writes (not common for a single-user client, more relevant for multi-device or collaborative scenarios), consider using these fields to avoid unintentionally overwriting data. The `applyWrites` batch endpoint can also bundle writes with a single `swapCommit` for atomicity [23].

Retries and Idempotency: Most record writes are **idempotent** if you use the same record key. For example, if you try to create a like record and you get a network failure, it's safe to retry (you might end up with a duplicate like which isn't allowed, but the server will likely return an error saying it already exists). Deleting a non-existent record will similarly error but is harmless. Use client-side UUIDs or timestamps as needed to ensure idempotency on your side as well.

Time Format: Timestamps in the protocol are ISO 8601 strings in UTC (e.g. `"2023-11-04T15:20:30.123Z"`). When providing `createdAt`, use `new Date().toISOString()`. The servers expect the `Z` (UTC). If your timestamps are significantly skewed from real time, some services might reject them (to prevent, say, posts from the far future or past).

Content Sizes: The lexicons often specify max lengths (e.g. posts max 300 characters or 256 graphemes). The client should enforce these before calling the API to avoid errors. For images, the `app.bsky.embed.images` lexicon says 1MB max per image [86]. Uploading a larger blob will fail with an error. So check file sizes and possibly downscale images on the client side.

Using the Official SDK vs Raw HTTP: It's recommended to use the official `@atproto/api` library for TypeScript/Node if possible. It covers a lot of details: proper headers, session management, serialization of inputs/outputs to the right types, etc. For example, it will ensure binary blobs are properly sent as bytes, and will parse the CBOR in binary frames if needed (for subscriptions). If using raw HTTP, make sure to set `Content-Type: application/json` on POSTs (except blob upload which should be the blob's mime type). Endpoints under `/xrpc/` don't do content negotiation – they expect JSON by default (except subscription which upgrades to WebSocket, and blob download which returns the actual blob bytes with its mime type).

Testing and Debugging: Bluesky provides an interactive API docs site (docs.bsky.app) where you can input parameters and execute requests if you have a bearer token. This can help in seeing what responses look like and what errors come back. Also, the `@atproto/api` library outputs some useful info if a request fails (it might throw an error object that includes `error` and `message` from the response).

Versioning: Keep an eye on Bluesky's announcements and the `bluesky-social/atproto` repo for changes to lexicons. New fields might appear (clients should ignore unknown fields in responses to be forward-compatible). Breaking changes are rare, but they might deprecate things like an endpoint in favor of another (e.g. if in future they unify some API). The lexicon IDs act as version identifiers; a change usually means a new NSID or a `#` fragment version bump.

Security: Use HTTPS for all endpoints. Validate inputs if you accept user text to avoid injecting into API calls. The AT Protocol uses JWTs for auth which have expiration – store refresh tokens securely if you need to persist sessions (they are like long-lived credentials). If building a multi-user app, do not mix up tokens between users – always keep them isolated per user account.

Community and Support: The AT Protocol is evolving, so some features (like DMs, or future group interactions) may change or get new lexicons. It's a good practice to join the developer community (Bluesky's dev Slack or forums, etc.) and follow updates. The official documentation at **atproto.com** and **docs.bsky.app** is the best source [87] [11], and the GitHub discussions/issues can also clarify behavior in edge cases.

With this comprehensive overview, you should be equipped to build on the Bluesky AT Protocol – from basic posting and social actions, to advanced feed algorithms and moderation-aware client features. Always refer back to the lexicon definitions for authoritative details, and happy coding with ATProto! [5] [32]

---

[1] [2] HTTP Reference | Bluesky
https://docs.bsky.app/docs/category/http-reference

[3] [6] [7] [8] [9] [10] [11] [12] OAuth Introduction - AT Protocol
https://atproto.com/guides/oauth

[4] [14] [15] [16] [22] Get Started | Bluesky
https://docs.bsky.app/docs/get-started

5 13 19 24 61 84 87 HTTP API (XRPC) - AT Protocol
https://atproto.com/specs/xrpc

17 18 20 32 33 34 35 36 37 38 39 86 7 posts tagged with "guide" | Bluesky
https://docs.bsky.app/blog/tags/guide

21 59 60 63 64 Quick start guide to building applications on AT Protocol - AT Protocol
https://atproto.com/guides/applications

23 com.atproto.repo.applyWrites | Bluesky
https://docs.bsky.app/docs/api/com-atproto-repo-apply-writes

25 26 27 28 30 Creating custom feeds in Bluesky: a data scientist's experience - Mark Torres Personal Website
https://markptorres.com/research/2023-11-29-working-with-bsky

29 65 66 67 68 69 70 Sync - AT Protocol
https://atproto.com/specs/sync

31 app.bsky.feed.getTimeline | Bluesky
https://docs.bsky.app/docs/api/app-bsky-feed-get-timeline

40 app.bsky.graph.muteActor | Bluesky
https://docs.bsky.app/docs/api/app-bsky-graph-mute-actor

41 com.atproto.moderation.createReport | Bluesky
https://docs.bsky.app/docs/api/com-atproto-moderation-create-report

42 43 44 45 46 47 48 49 50 51 54 55 56 57 58 Labels - AT Protocol
https://atproto.com/specs/label

52 Unable to update feed preferences · Issue #830 - GitHub
https://github.com/bluesky-social/social-app/issues/830

53 https://cdn.jsdelivr.net/npm/@atproto/pds@0.4.200/...
https://cdn.jsdelivr.net/npm/@atproto/pds@0.4.200/tests/preferences.test.ts

62 75 76 77 78 Handle - AT Protocol
https://atproto.com/specs/handle

71 72 80 81 Identity - AT Protocol
https://atproto.com/guides/identity

73 com.atproto.identity.resolve_handle - The AT Protocol SDK
https://atproto.blue/en/latest/atproto/atproto_client.models.com.atproto.identity.resolve_handle.html

74 Resolving Identities - Bluesky Documentation
https://docs.bsky.app/docs/advanced-guides/resolving-identities

79 com.atproto.identity.resolveHandle - Bluesky Documentation
https://docs.bsky.app/docs/api/com-atproto-identity-resolve-handle

82 83 chat.bsky.convo.sendMessage | Bluesky
https://docs.bsky.app/docs/api/chat-bsky-convo-send-message

85 com.atproto.repo.create_record - The AT Protocol SDK
https://atproto.blue/en/latest/atproto/atproto_client.models.com.atproto.repo.create_record.html